

# GYG System Design Playbook — Booking / Inventory

---

Bu dosya, arkadaşının Ticketmaster referansını GetYourGuide domain diline çevirir.

Ana guide: [README.md](#) §6.

---

## 1. Neden bu domain?

GetYourGuide'ın ürünü: dünya çapında experience / tour booking.

Mühendislik blog'ları sürekli şunları konuşuyor:

- Availability & pricing for 200k+ activities
- Supplier / reservation system integrations
- Checkout'ta "available → sold out" tutarsızlığı
- PHP → Java migrations, dual-write, Kafka/Debezium
- Explicit domain events, GraphQL reads, feature-flag rollouts

Yani "generic URL shortener"dan ziyade **contention'lı booking** seni bekliyor.

Kaynaklar:

- <https://getyourguide.careers/posts/from-distributed-monolith-to-clear-boundaries-redesigning-microservices-at-getyourguide>
  - <https://getyourguide.careers/posts/boxoffice-migration>
  - <https://www.hellointerview.com/learn/system-design/problem-breakdowns/ticketmaster>
- 

## 2. Interview script (yüksek sesle pratik)

### 2.1 Opening (2 dk)

"GYG tarzı bir experience booking sistemi tasarlayacağımı varsayıyorum: traveler bir tour'u görür, arar, müsait slot için rezervasyon yapar. Doğru mu? Ödeme sağlayıcısı detayı ve admin panelini out-of-scope bırakabilir miyiz?"

### 2.2 Functional requirements

Above the line:

1. View experience/tour + availability
2. Search / browse
3. Reserve + book (no double booking)

Below:

- Reviews, personalization, dynamic pricing, fraud, multi-currency details (istersen "later" de)

## 2.3 Non-functional

- Booking path: **strong consistency** on inventory units
- Catalog/search: high availability, eventual OK for non-critical fields
- Spike: viral tour / limited museum slot
- Read-heavy browsing
- p99 search latency target (ör. <300–500ms)
- Supplier API flaky → graceful degradation

## 2.4 Entities

Experience / Tour  
Variant (language, group size, ...)  
AvailabilitySlot (start, capacity, remaining)  
Supplier  
Booking (state: CREATED|HELD|CONFIRMED|CANCELLED|EXPIRED)  
BookingItem  
PaymentIntent (external)  
Customer

GYG blog kavramı: computed **isBookable** — supplier status, tour status, variant status, integration health, upcoming slots, manual override.

## 2.5 APIs

GET /v1/experiences/{id}  
GET /v1/experiences/{id}/availability?from&to  
GET /v1/search?q&city&date&page  
POST /v1/holds {variantId, slotId, qty, idempotencyKey}  
POST /v1/bookings {holdId, paymentToken, idempotencyKey}  
DELETE /v1/holds/{id} # optional cancel/release

Neden hold + book ayrımı?

Payment ve kullanıcı tereddüdü süresince inventory'yi **TTL ile kilitlemek** için.

---

## 3. High-level architecture (çiz)

Client → API Gateway →  
Catalog/Search Service → (ES/OpenSearch + read replicas)  
Inventory Service → (MySQL/Postgres primary; remaining seats)  
Booking Service → (bookings DB; state machine)  
Payment Service → (PSP)  
Connectivity Adapters → (supplier APIs; no business SoT)  
↓  
Kafka (availability\_updated, booking\_confirmed, supplier\_deactivated)

**Senior cümlesi:** "Connectivity protokol çevirir; business truth Inventory'de."

---

## 4. Deep dive A — Double booking

### Seenekler

| Yaklaşım                                                      | Artı              | Eksi               |
|---------------------------------------------------------------|-------------------|--------------------|
| UPDATE ... SET remaining = remaining - n WHERE remaining >= n | Basit, atomic     | Hot row contention |
| Optimistic version column                                     | Daha az lock wait | Retry logic        |
| Per-slot row lock / SELECT FOR UPDATE                         | Güçlü             | Spike'te latency   |
| sharded counters / seat rows                                  | Scale             | Complexity         |

### Hold modeli:

1. Transaction: remaining kontrol + hold kaydı + remaining düş
2. TTL (ör. 10 dk) — scheduler veya delay queue ile expire
3. Confirm: hold valid mi? payment OK mu? booking CONFIRMED
4. Expire: remaining geri ekle (idempotent)

**Idempotency:** client Idempotency-Key → aynı hold/booking dön.

---

## 5. Deep dive B — Search vs checkout consistency

GYG'nin anlattığı fail: search “available”, checkout “sold out” çünkü her servis kendi cache flag'ine bakıyor.

### Cevabın:

- Search index **yaklaşık** availability gösterebilir (eventual)
  - Checkout **Inventory.isBookable + remaining** otoritesine bakar
  - UI copy: “availability may change” / hold sırasında kesinleşir
  - Cache invalidation: availability\_updated event → search worker
- 

## 6. Deep dive C — Supplier flakiness

- Circuit breaker + timeouts
  - Cached last-known availability with explicit staleness
  - Degrade: “request to supplier” / hide book CTA when integration unhealthy
  - Alert on isBookable false transitions with reason snapshot (blog'daki audit trail)
-

## 7. Deep dive D — Migration / dual-write (bonus Senior points)

BoxOffice → Inventory pattern:

1. Dual write / CDC (Debezium → Kafka)
2. Flip reads (feature flag, endpoint-by-endpoint)
3. Flip writes
4. Delete old

Metrics: 4xx/5xx, mismatch logs (shadow mode), booking conversion.

Mülakatta: “Big bang yapmam; shadow + % rollout.”

---

## 8. Observability checklist (söyle)

- RED/USE on booking & inventory
  - SLO: booking success rate, hold → confirm conversion, p99 reserve latency
  - Business: sold-out after search rate
  - Event lag (Kafka)
  - Supplier error budget
- 

## 9. Engineering Principles'ı cevaba yedir

| Principle                          | Design'da nasıl görünür                |
|------------------------------------|----------------------------------------|
| Simple services, clean interfaces  | Inventory vs Booking vs Connectivity   |
| Clarity over cleverness            | Hold state machine açık                |
| Proven technologies                | Postgres + Kafka + ES; exotic CRDT yok |
| Detect failures before they matter | Shadow mode, canaries                  |
| Expect failures                    | Supplier degrade                       |
| Informed ops                       | SLOs                                   |
| Done > perfect                     | Search eventual, book strong           |
| Leave better                       | Explicit events instead of triggers    |

---

## 10. 5 mock prompt (kendine sor)

1. Design GetYourGuide availability + booking for a limited museum slot tour.
2. Design ticket service for a stadium (Ticketmaster classic).
3. How would you migrate booking writes from PHP service to Java without downtime?
4. Search says 4 seats left; 500 users click Book — walk through the race.

5. Supplier API starts returning 15s latency — what do you change in 1 hour / 1 week / 1 quarter?

Her birini 30–45 dk'da çiz; kaydet; zayıf deep dive'ı tekrarla.

---

## 11. Minimal data model sketch

```
availability_slot(  
  id, variant_id, starts_at, capacity, remaining, version, updated_at  
)  
  
hold(  
  id, slot_id, qty, customer_id, expires_at, status, idem_key UNIQUE  
)  
  
booking(  
  id, hold_id, status, total, created_at, idem_key UNIQUE  
)
```

Confirm transaction (pseudo):

```
BEGIN;  
  SELECT hold FOR UPDATE;  
  assert hold.active && now < expires;  
  UPDATE booking ... CONFIRMED;  
  -- remaining already decremented at hold time  
COMMIT;
```

Expire job:

```
UPDATE hold SET status='EXPIRED' WHERE expires_at < now AND status='ACTIVE' RETURNING *;  
-- for each: UPDATE slot SET remaining = remaining + qty
```

---

## 12. Closing line (interview)

“Browse path'te availability için optimize ederim; money path'te Inventory'yi tek source of truth yapıp hold+idempotency ile double-booking'i kapatırım. Supplier kenarını adapter + degrade ile izole ederim.”